Lab

Interfaces

Lab: Object-oriented code — interfaces

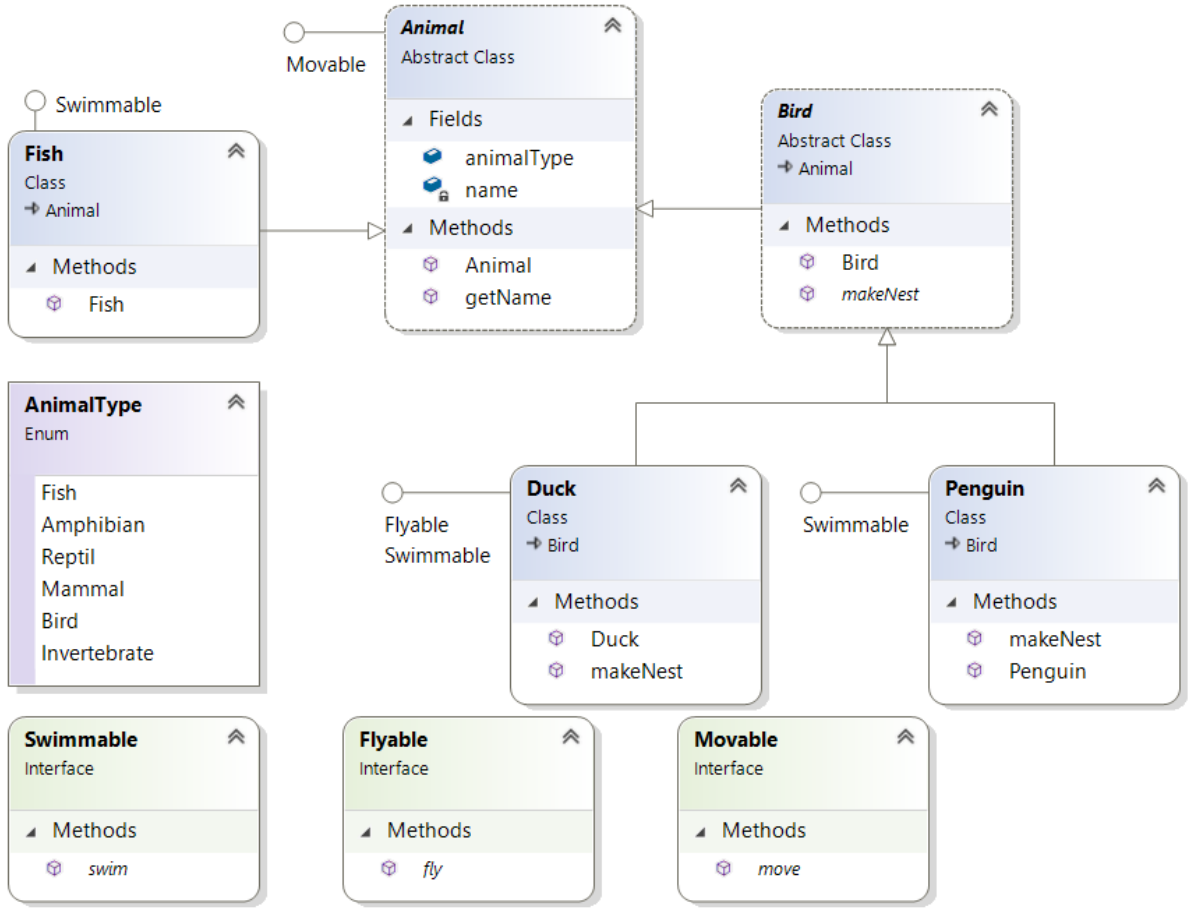
Objective

The primary objective of this lab is to provide you with the skills necessary to be able to:

- Define and implement interfaces.

Overview

In this lab you will implement a few interfaces to practice this subject. The class diagram after completing the lab can be seen below.



Step-by-step

1. Following the code you generated in the previous lab, add the following interfaces to the package.
    a) **Swimmable** with a method called swim()

b) **Flyable** with a method called fly()

c) Movable with a method called move()

See example below for a movable interface:

```
public interface Movable {
        void move();
}
```

2. Please view the class diagram above and then implement the interfaces.

   Just display a message for every implemented method. For example, in the Duck class:

```
public void move {
  System.out.println("Moving like a Duck!");
}

public void fly {
  System.out.println("Flying like a Duck!");
}
```

3. Open the class `Program` and write a few lines of code to test the interfaces' code.

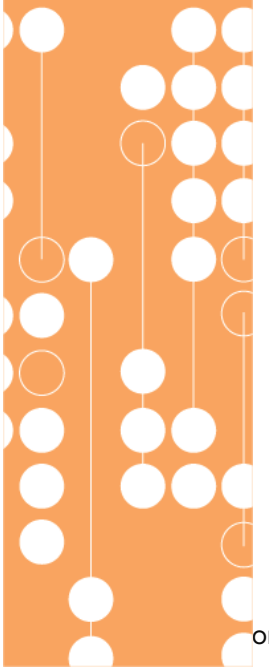
   a) In the '`for`' loop that iterates over the **animals** array, write an `if` statement to test if an Animal object is `Flyable` and if it is, cast it as **Flyable** and then call its **fly()**.

4. Run your code to make sure it works.

5. Inside the loop, test `if` the Animal object is **Swimmable** and if it is, cast it as Swimmable in order to call its swim()method.

Optional lab

Generic interfaces



Lab: Generic interfaces

Objective

In this lab, you will use and understand the role of generic interfaces.

Overview

In Java, **generic interfaces** are interfaces that use generics to specify the types they operate on. This allows them to work with different data types. In this lab you will experience using generic interface in the same way that you used generic classes like List<Car>.

Step by step

1. Use the **Account** class you created earlier and create an ArrayList<Account> called accounts.
2. Add a few Account instances to the ArrayList.like:

```
ArrayList<Account> accounts = new ArrayList<>( );
accounts.add(new Account(100, "Bob", 1000));
accounts.add(new Account(500, "Linda", 3000));
accounts.add(new Account(300, "David", 2000));
Collections.sort(accounts);
```

Your code will fail to compile because the system cannot determine the criteria by which you intend to sort the accounts – whether it's by ID, balance, or the owner's name. Let's fix this problem by using one of Java's Generic Interfaces.

3. Change the Account class code to implement Java's generic Comparable interface like:

```
class Account implements Comparable<Account>{

}
```

Press Ctrl-1 to implement the interface as:

```
  @Override
  public int compareTo(Account o) {
          // TODO Auto-generated method stub
          return 0;
}
```

This method returns 0 if the current object and Account parameter are equal. It returns a positive value is bigger and negative if smaller. For example, to compare two Accounts by their balance type:

```
   @Override
      public int compareTo(Account other) {
         return (int)(this.balance - other.balance);
}
```

Your code should now compile, and you test it by displaying the account objects. Try sorting Accounts using another criteria, such as the owner's name.

Learn. To Change.

QA.com